



KYAMBOGO UNIVERSITY

**BACHELOR OF ENGINEERING IN MECHATRONICS AND
BIOMEDICAL ENGINEERING.**

TEMB 1202: COMPUTING II

ASSIGNMENT: EMPLOYEE PAYROLL SYSTEM

DATE: 27TH MAY, 2026

GROUP D EVENING

NAME	REGISTRATION NUMBER
KATONGOLE SHADDAI KISA	25/U/BIE/01380/PE
ZALWANGO CHRISTINE	25/U/BIE/01426/PE
MUTYABA DERICK	25/U/BIE/01399/PE
NAKANWAGI MARIAM	25/U/BIE/01403/PE
BUJINGO DAVID KABANDA	25/U/BIE/01364/PE
RWOTOMIA PIUS EMMANUEL	25/U/BIE/01419/PE

INTRODUCTION

This report documents the design and implementation of an Employee Payroll System written in the C programming language. The system is a fully menu-driven console application that enables payroll officers to manage employee records, compute salaries with overtime, apply statutory deductions, and generate comprehensive payroll reports. It demonstrates practical application of C programming concepts including structures, arrays, functions, pointers, loops, and conditional logic. The program runs continuously until the user selects Exit, and requires login authentication before any payroll operations can be performed. All inputs are validated against defined business rules to prevent incorrect salary calculations

PROBLEM STATEMENT

An organization needs a digital payroll system to replace manual salary processing. The system must handle the complete payroll lifecycle from registering employees to computing gross pay, applying deductions, calculating net pay, and producing summary reports while enforcing the company's payroll rules.

Required Operations

- Register new employees with name, department, hourly rate, and hours worked.
- Search existing employee records by name or employee ID.
- Update employee details including rate, hours worked, and department.
- Delete employee records for staff who have left the organization.
- Calculate salaries, gross pay, overtime, deductions, and net pay.
- Display a complete payroll summary report identifying the highest-paid employee.

Business rules

Normal hours limit	Normal working hours are capped at 160 hours/month.
Overtime pays	Hours above 160 are paid at 1.5* the base hourly rate.
Excessive hours warning	Hours above 300 trigger a visible warning message.
Deductions on Gross pay	Tax 30%, NSSF 5%, Health insurance 2% of gross pay.
Negative input rejection	Negative hours or rates are rejected with an error.
Highest-paid employee	The report identifies the employee with highest net pay.

SYSTEM FEATURES AND FUNCTIONALITY

Authentication On launch the system presents a login prompt. After three consecutive failed attempts access is denied and the program exits, preventing unauthorized access.

Main menu

Feature	Description
Registration employee	Add a new employee record
Search employee	Find by name or ID
Update employee	Modify rate, hours, or department
Delete employee	Remove a record permanently
Calculate all salaries	Refresh payroll for all employees
Display payroll report	Full summary + highest-paid employee
Exit	Logout and close the system

Salary Calculation Steps

Step	Item	Formula
Step 1	Normal pay	Hours $\leq$ 160 - Hours $\times$ Rate
Step 2	Overtime pays	(Hours - 160) $\times$ Rate $\times$ 1.5
Step 3	Gross pay	Normal pay + Overtime pay
Step 4	Tax	Gross $\times$ 30%
Step 5	NSSF	Gross $\times$ 5%
Step 6	Health ins	Gross $\times$ 2%
Step 7	Total deductions	Tax + NSSF + Health insurance
Step 8	Net pay	Gross pay - Total deductions

PROGRAM DESIGN AND STRUCTURE

Data Structure

The central data structure is a C **struct** called **Employee**. An array of up to MAX_EMPLOYEES (100) of this struct acts as the in-memory database.

```
typedef struct {
    int empID;
    char name[60];
    char department[40];
    float hourlyRate;
    float hoursWorked;
    float normalPay;
    float overtimePay;
    float grossPay;
    float taxAmount;
    float nssfAmount;
    float healthAmount;
    float totalDeductions;
    float netPay;
    int overtimeFlag;
    int excessiveFlag;
```

```
} Employee;
```

Constants

```
#define MAX_EMPLOYEES 100
```

```
#define NORMAL_HOURS 160.0f
```

```
#define EXCESSIVE_HOURS 300.0f
```

```
#define OT_MULTIPLIER 1.5f
```

```
#define TAX_RATE 0.30f
```

```
#define NSSF_RATE 0.05f
```

```
#define HEALTH_RATE 0.02f
```

```
#define ID_BASE 100
```

IMPORTANT FUNCTIONS USED

```
login()
```

Signature: void login(void)

Handles authentication. Allows 3 attempts then exits. First function called in main().

```
registerEmployee()
```

Signature: void registerEmployee(void)

Collects employee data, validates negatives, auto-assigns ID, calls calculateSalary().

```
searchEmployee()
```

Signature: void searchEmployee(void)

Finds by case-insensitive name (partial match) or exact ID. Displays full record.

```
updateEmployee()
```

Signature: void updateEmployee(void)

Updates rate, hours, department, or all fields. Recalculates salary after change.

```
deleteEmployee()
```

Signature: void deleteEmployee(void)

Confirms deletion, removes record by array shifting, decrements count.

```
calculateSalary()
```

Signature: void calculateSalary(Employee *emp)

Core computation: normal pay, overtime, gross pay, all deductions, net pay, flags.

`calculateAllSalaries()`

Signature: `void calculateAllSalaries(void)`

Loops through all employees and calls `calculateSalary()` on each record.

`displayPayrollReport()`

Signature: `void displayPayrollReport(void)`

Prints formatted table, totals, highest-paid employee, and deduction rates used.

`displayEmployee()`

Signature: `void displayEmployee(Employee *emp)`

Prints a single employee's complete payroll breakdown neatly.

`findByID()`

Signature: `int findByID(int id)`

Linear search returning array index for matching `empID`, or -1 if not found.

`findByName()`

Signature: `int findByName(const char *query)`

Case-insensitive partial string match using `strstr()`. Returns first matching index.

`clearBuffer()`

Signature: `void clearBuffer(void)`

Flushes `stdin` after `scanf()` to prevent stale newline issues.

`toLowerCase()` Signature: `void toLowerCase(char *str, char *out)`

Converts string to lowercase for case-insensitive comparisons.

`printDivider()`

Signature: `void printDivider(char ch, int len)`

Prints a horizontal line of a given character — used for consistent UI formatting

DESIGN DECISIONS AND JUSTIFICATIONS

Storing All Computed Fields in the Struct

`grossPay`, `deductions`, `netPay`, `overtimeFlag`, and `excessiveFlag` are stored in the `Employee` struct rather than recalculated on demand. This ensures every display function reads consistent values and avoids duplicating the calculation logic.

`#define` Constants for All Rates

Using named constants (`TAX_RATE`, `OT_MULTIPLIER`, etc.) rather than magic numbers makes the code self-documenting. Changing a statutory rate

requires editing one line only, which is essential for payroll software where rates change annually.

`calculateSalary()` Called at Registration and After Updates

Isolating the calculation in its own function and calling it both on registration and after every update guarantees that stored figures are always current. Without this, updating an hourly rate would leave stale net pay values.

Employee IDs Starting at 1001 Starting IDs at 1001 produces professional-looking employee numbers clearly distinct from array indices, reducing off-by-one confusion.

`fgets` for Name Input

`fgets()` reads full lines including spaces, so "John Doe" is captured correctly instead of being truncated at the space by `scanf("%s")`.

Global Array Instead of Linked List

An array is simpler to implement using the C concepts covered in this course, provides $O(1)$ indexed access, and is adequate for up to 100 employees. A linked list would be preferable for very large, dynamic datasets

CHALLENGES ENCOUNTERED

Reading Names with Spaces

`scanf("%s")` stops at spaces. We switched to `fgets()` and stripped the trailing newline with `strcspn()` to correctly capture full names like "John Doe".

Stale Salary After Updates

Early versions did not recalculate salary after an update. We fixed this by always calling `calculateSalary()` at the end of `updateEmployee()`.

Input Buffer Contamination

Mixing `scanf()` for numbers and `fgets()` for strings caused empty reads. A `clearBuffer()` helper that consumes remaining characters resolved this.

Case-Insensitive Search

The standard `strcmp()` is case-sensitive. We implemented `toLowerCase()` and used `strstr()` to support partial, case-insensitive name searches.

Safe Report When List is Empty

Accessing `employees[0]` when `empCount` is 0 causes undefined behaviour. We added a guard (if `empCount == 0`) before entering the report loop.

GitHub Merge Conflicts

Multiple members editing the same struct definition caused conflicts. We resolved this by agreeing on the final struct layout first and committing it before branching for individual functions

CONCLUSION

The Employee Payroll System was successfully designed, implemented, and tested as a fully functional menu-driven C program satisfying all requirements of the assignment. All business rules were enforced: normal hours are capped at 160/month, overtime is paid at $1.5\times$ the base rate, hours above 300 trigger a visible warning, deductions (tax 30%, NSSF 5%, health 2%) are computed on gross pay, negative inputs are rejected, and the payroll report correctly identifies the highest-paid employee. This project provided valuable hands-on experience translating real-world business requirements into working software, reinforced core C programming skills, and built collaborative development habits through GitHub version control.